INSTITUTO POLITÉCNICO NACIONAL

Unidad Profesional Interdisciplinaria en Ingeniería y Tecnologías Avanzadas
UPIITA

Asignatura: Aplicaciones Distribuidas

Profesor: Noe Sierra Romero

Práctica: Práctica de repaso para API Restful

Alumna: Bernal Aguilar Yuvia Abigail

Código:

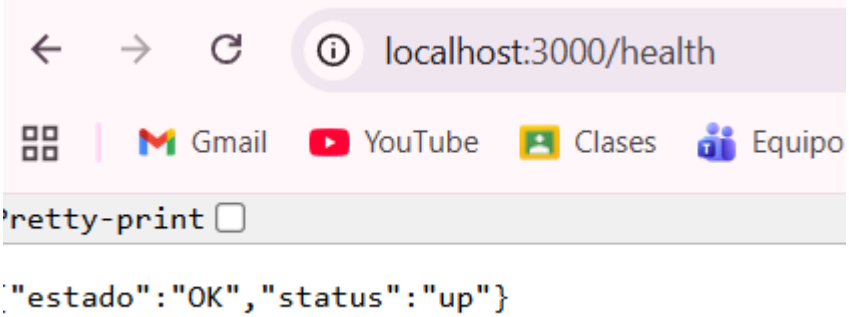
```
const express = require("express");

const app = express();
app.use(express.json());

// Helpers
const isStr  = (x) => typeof x === "string";
const isNum  = (x) => typeof x === "number" && Number.isFinite(x);
const isBool = (x) => typeof x === "boolean";
const isInt  = (x) => Number.isInteger(x);
const normScale = (s) => (isStr(s) ? s.trim().toUpperCase() : s);

const ok   = (res, data = {}, code = 200) => res.status(code).json(data);
const fail = (res, error, code = 400)     => res.status(code).json({ error });

// Captura JSON invalido
app.use((err, req, res, next) => {
  if (err instanceof SyntaxError && "body" in err)
    return fail(res, "JSON inválido en el body");
  next();
});

// Ejercicio 1: /saludo
app.post("/saludo", ({ body }, res) => {
  const { nombre } = body ?? {};
  if (!isStr(nombre) || !nombre.trim())
    return fail(res, "'nombre' debe ser string no vacío.");
  return ok(res, { mensaje: `Hola, ${nombre}` });
});

// Ejercicio 2: /calcular
const OPS = {
  suma:          (a, b) => a + b,
  resta:         (a, b) => a - b,
  multiplicacion: (a, b) => a * b,
  division:      (a, b) => a / b,
};

app.post("/calcular", ({ body }, res) => {
  const { a, b, operacion } = body ?? {};
  if (!isNum(a) || !isNum(b) || !isStr(operacion))
    return fail(res, "Se esperan { a: number, b: number, operacion: string }.");

  const op = operacion.trim().toLowerCase();
  if (!OPS[op])
    return fail(res, "Operación inválida. Use: suma | resta | multiplicacion | division.");
  if (op === "division" && b === 0)
    return fail(res, "División por cero no permitida.");
```

```
  return ok(res, { resultado: OPS[op](a, b) });
});

// Ejercicio 3: /tareas (CRUD en memoria)
let tareas = [];

app.post("/tareas", ({ body }, res) => {
  const { id, titulo, completada } = body ?? {};
  if (!isInt(id) || !isStr(titulo) || !isBool(completada))
    return fail(res, "Se esperan { id: integer, titulo: string, completada: boolean }.");
  if (tareas.some((t) => t.id === id))
    return fail(res, "Ya existe una tarea con ese id.");

  const nueva = { id, titulo, completada };
  tareas.push(nueva);
  return ok(res, { tarea: nueva }, 201);
});

app.get("/tareas", (_, res) => ok(res, { tareas }));

app.put("/tareas/:id", ({ params, body }, res) => {
  const id = Number(params.id);
  if (!isInt(id)) return fail(res, ":id debe ser entero.");

  const idx = tareas.findIndex((t) => t.id === id);
  if (idx === -1) return fail(res, "Tarea no encontrada.", 404);

  const { titulo, completada } = body ?? {};
  if (titulo !== undefined && (!isStr(titulo) || !titulo.trim()))
    return fail(res, "'titulo' debe ser string no vacío.");
  if (completada !== undefined && !isBool(completada))
    return fail(res, "'completada' debe ser boolean.");

   tareas[idx] = { ...tareas[idx], ...(titulo !== undefined && { titulo }), ...(completada !==
undefined && { completada }) };
  return ok(res, { tarea: tareas[idx] });
});

app.delete("/tareas/:id", ({ params }, res) => {
  const id = Number(params.id);
  if (!isInt(id)) return fail(res, ":id debe ser entero.");

  const idx = tareas.findIndex((t) => t.id === id);
  if (idx === -1) return fail(res, "Tarea no encontrada.", 404);

  return ok(res, { tarea: tareas.splice(idx, 1)[0] });
});

// Ejercicio 4: /validar-password
const PASSWORD_RULES = [
  [/.{8,}/,  "Debe tener mínimo 8 caracteres."],
```

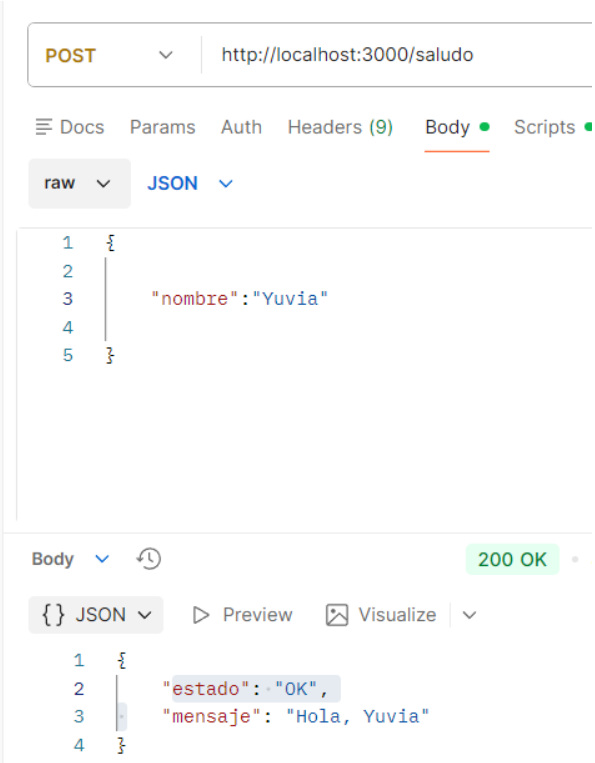
```
  [/[A-Z]/,  "Debe incluir al menos una mayúscula."],
  [/[a-z]/,  "Debe incluir al menos una minúscula."],
  [/[0-9]/,  "Debe incluir al menos un número."],
];

app.post("/validar-password", ({ body }, res) => {
  const { password } = body ?? {};
  if (!isStr(password)) return fail(res, "'password' debe ser string.");

  const errores = PASSWORD_RULES.filter(([re]) => !re.test(password)).map(([, msg]) =>
msg);
  return ok(res, { esValida: errores.length === 0, errores });
});

// Ejercicio 5: /convertir-temperatura
const toKelvin   = { C: (v) => v + 273.15, F: (v) => (v - 32) * (5 / 9) + 273.15, K: (v) => v };
const fromKelvin = { C: (k) => k - 273.15, F: (k) => (k - 273.15) * (9 / 5) + 32,  K: (k) => k
};

app.post("/convertir-temperatura", ({ body }, res) => {
  const { valor, desde, hacia } = body ?? {};
  const d = normScale(desde);
  const h = normScale(hacia);

  if (!isNum(valor) || !toKelvin[d] || !fromKelvin[h])
    return fail(res, "Se esperan { valor: number, desde: C|F|K, hacia: C|F|K }.");

  return ok(res, {
    valorOriginal:    valor,
    valorConvertido:  fromKelvin[h](toKelvin[d](valor)),
    escalaOriginal:   d,
    escalaConvertida: h,
  });
});

// Ejercicio 6: /buscar
app.post("/buscar", ({ body }, res) => {
  const { array, elemento } = body ?? {};
  if (!Array.isArray(array)) return fail(res, "'array' debe ser un arreglo.");

  const indice = array.findIndex((x) => Object.is(x, elemento));
  return ok(res, { encontrado: indice !== -1, indice, tipoElemento: typeof elemento });
});

// Ejercicio 7: /contar-palabras
app.post("/contar-palabras", ({ body }, res) => {
  const { texto } = body ?? {};
  if (!isStr(texto)) return fail(res, "'texto' debe ser string.");

  const palabras = texto.trim() ? texto.trim().split(/\s+/) : [];
  return ok(res, {
```

```
    totalPalabras:   palabras.length,
    totalCaracteres: texto.length,
    palabrasUnicas:  new Set(palabras.map((p) => p.toLowerCase())).size,
  });
});

// Health & 404
app.get("/health", (_, res) => ok(res, { status: "up" }));
app.use((req, res) => fail(res, "Ruta no encontrada.", 404));

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`API corriendo en http://localhost:${PORT}`));
```

Usando el http://localhost:3000/health, observamos que esta funcionando el servicio:

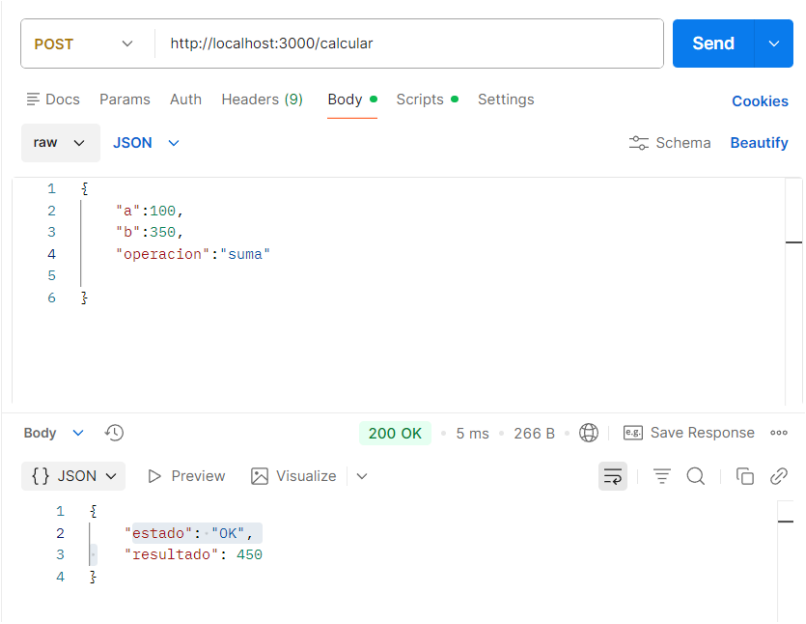


1. Servicio de saludo básico:



2. Calculadora de operaciones básicas:

- Suma



- Resta

```
{
    "a":100,
    "b":350,
    "operacion":"resta"

}
```

Body 200 OK

{} JSON Preview Visualize

```
{
    "estado": "OK",
    "resultado": -250
}
```

- Multiplicación

```
{
    "a":100,
    "b":350,
    "operacion":"multiplicacion"

}
```

Body 200 OK 4 m

{} JSON Preview Visualize

```
{
    "estado": "OK",
    "resultado": 35000
}
```

- División

```
{
    "a":100,
    "b":350,
    "operacion":"division"

}
```

Body 200 OK

{} JSON Preview Visualize

```
{
    "estado": "OK",
    "resultado": 0.2857142857142857
}
```

3. Gestor de tareas (CRUD Básico)

- Primero creamos tarea en POST

POST http://localhost:3000/tareas

Docs Params Auth Headers (9) Body Scripts Setting

raw JSON

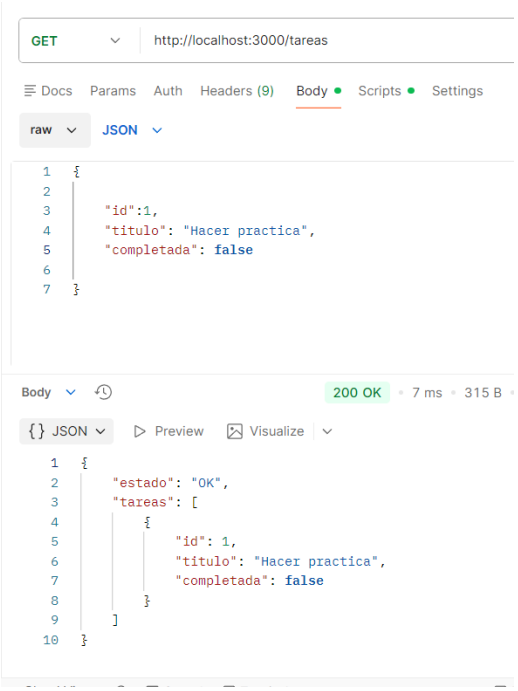
```
{

    "id":1,
    "titulo": "Hacer practica",
    "completada": false

}
```

Body 201 Created 4 ms 3

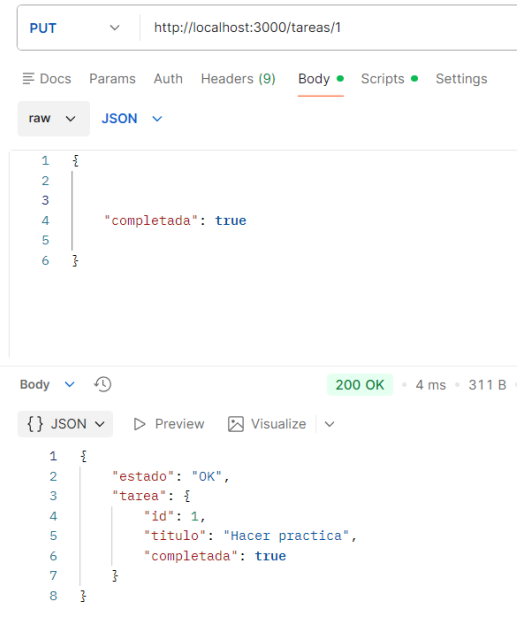
{} JSON Preview Visualize

```
{
    "estado": "OK",
    "tarea": {
        "id": 1,
        "titulo": "Hacer practica",
        "completada": false
    }
}
```

- Listamos las tareas con GET



Actualizamos tarea con PUT



4. Validador de contraseñas:

POST http://localhost:3000/validar-password

Docs Params Auth Headers (9) Body Scripts Settings

raw JSON

```
{
    "password":"hola123"
}
```

Body 200 OK 6 ms 355 B

{} JSON Preview Visualize

```
{
    "estado": "OK",
    "esValida": false,
    "errores": [
        "Debe tener mínimo 8 caracteres.",
        "Debe incluir al menos una mayúscula."
    ]
}
```

POST http://localhost:3000/validar-password

Docs Params Auth Headers (9) Body Scripts Settin

raw JSON

```
{
    "password":"Contraseña11"
}
```

Body 200 OK 4 ms 2

{} JSON Preview Visualize

```
{
    "estado": "OK",
    "esValida": true,
    "errores": []
}
```

5. Conversor de temperaturas:

POST http://localhost:3000/convertir-temperatura

Docs Params Auth Headers (9) Body Scripts Settir

raw JSON

```
{
    "valor":32,
    "desde":"C",
    "hacia":"F"
}
```

Body 200 OK 4 ms

{} JSON Preview Visualize

```
{
    "estado": "OK",
    "valorOriginal": 32,
    "valorConvertido": 89.6,
    "escalaOriginal": "C",
    "escalaConvertida": "F"
}
```

6. Buscador en Array:

POST http://localhost:3000/buscar

Docs Params Auth Headers (9) Body Scripts

raw JSON

```
{
    "array": [11 ,9, 3, 5],
    "elemento":5
}
```

Body 200 OK

{} JSON Preview Visualize

```
{
    "estado": "OK",
    "encontrado": true,
    "indice": 3,
    "tipoElemento": "number"
}
```

POST http://localhost:3000/buscar

Docs Params Auth Headers (9) Body Scripts

raw JSON

```
1 {
2
3
4     "array": [11 ,9, 3, 5],
5     "elemento":8
6
7
8
9 }
```

Body 200 OK

{} JSON Preview Visualize

```
1 {
2     "estado": "OK",
3     "encontrado": false,
4     "indice": -1,
5     "tipoElemento": "number"
6 }
```

POST http://localhost:3000/buscar

Docs Params Auth Headers (9) Body Scripts

raw JSON

```
{
    "array": ["y" ,"b", "a", "m"],
    "elemento":"a"
}
```

Body 200 OK

{} JSON Preview Visualize

```
{
    "estado": "OK",
    "encontrado": true,
    "indice": 2,
    "tipoElemento": "string"
}
```

7. Contador de palabras:

POST http://localhost:3000/contar-palabras Send

Docs Params Auth Headers (9) Body Scripts Settings Cook

raw JSON Schema Beaut

```
{
    "texto": "Estoy haciendo la segunda practica de aplicaciones distribuidas"
}
```

Body 200 OK 5 ms 308 B Save Response

{} JSON Preview Visualize

```
{
    "estado": "OK",
    "totalPalabras": 8,
    "totalCaracteres": 63,
    "palabrasUnicas": 8
}
```

POST http://localhost:3000/contar-palabras

Docs Params Auth Headers (9) Body Scripts Set

raw JSON

```
1 {
2
3
4     "texto": "Mi cama es rosa y mi mochila es ros
5
6
7
8 }
```

Body 200 OK 5 ms

{} JSON Preview Visualize

```
1 {
2     "estado": "OK",
3     "totalPalabras": 9,
4     "totalCaracteres": 36,
5     "palabrasUnicas": 6
6 }
```